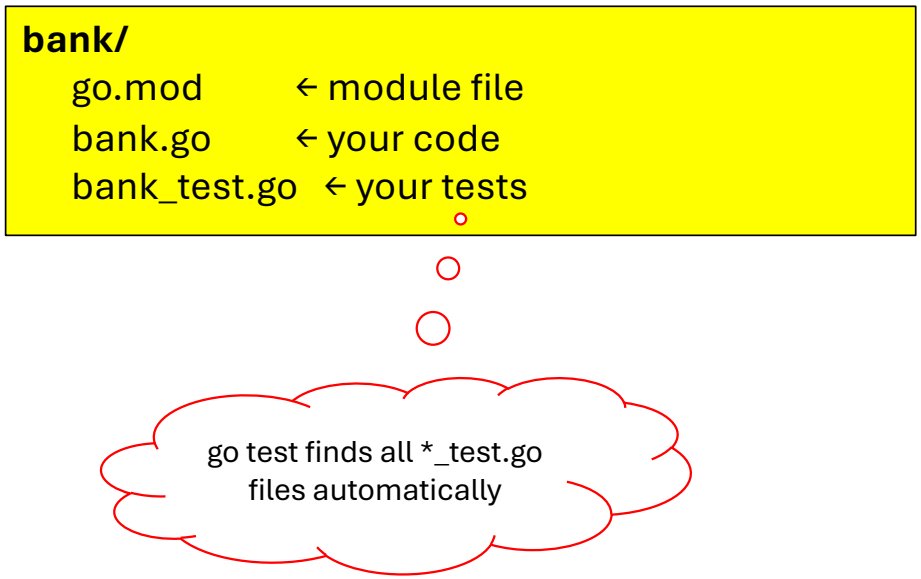
Juniorprogrammierer.de

Go 5: Testing

2024/25/26 – Sascha Stojanovic

Testing in Go = built in

- A test is just Go code that checks your Go code
- Unlike Java (JUnit) you don't need to install anything
- Three rules:
  - File name ends with **_test.go**
  - Same package as the code you test
  - Function starts with **Test** and gets **t *testing.T**
- Run all tests in your folder with **go test**
- Why test? Find bugs early and change code without fear



Your first test

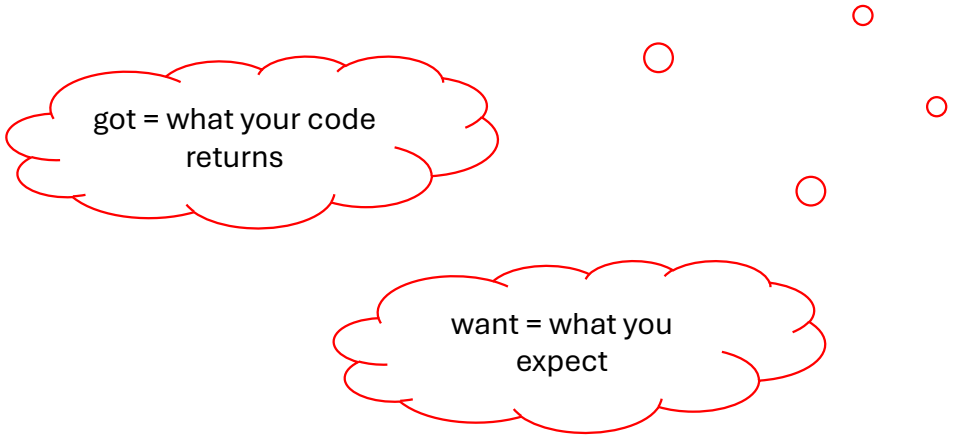
```
// bank.go
package bank

func Deposit(balance, amount int) int {
    return balance + amount
}
```

```
// bank_test.go
package bank

import "testing"

func TestDeposit(t *testing.T) {
    got := Deposit(100, 50)
    want := 150
    if got != want {
        t.Errorf("got %d, want %d", got, want)
    }
}
```



Running tests

- **go test** → short result
- **go test -v** → shows every single test
- **go test -run TestDeposit** → runs only this test
- What happens if you change **want := 150** to **want := 200**?

```
$ go test -v
=== RUN   TestDeposit
--- PASS: TestDeposit (0.00s)
PASS
ok      example.com/bank   0.002s
```

```
$ go test
--- FAIL: TestDeposit (0.00s)
    bank_test.go:9: got 150, want 200
FAIL
FAIL    example.com/bank   0.002s
```

File + line of the failing check

Table-driven tests = the Go way

```
func TestDeposit(t *testing.T) {
    tests := []struct {
        name    string
        balance int
        amount  int
        want    int
    }{
        {"normal", 100, 50, 150},
        {"zero amount", 100, 0, 100},
        {"empty account", 0, 30, 30},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            got := Deposit(tt.balance, tt.amount)
            if got != tt.want {
                t.Errorf("got %d, want %d", got, tt.want)
            }
        })
    }
}
```

One line = one test case

t.Run = named sub-test

- New test case? Just add one line
- Every case shows up on its own in **go test -v**

Testing errors

- Test the happy path AND the error path
- **err == nil** → no error happened
- **errors.As** → checks if the error is your custom type (e.g. AgeError from Go 4) and gives you its fields
- Why is it dangerous to test only the happy path?

```
func TestDivideByZero(t *testing.T) {
    _, err := divide(10, 0)
    if err == nil {
        t.Fatal("expected an error, got nil")
    }
}

func TestCheckAge(t *testing.T) {
    err := checkAge(15)
    var ageErr AgeError
    if !errors.As(err, &ageErr) {
        t.Fatalf("expected AgeError, got %v", err)
    }
    if ageErr.Age != 15 {
        t.Errorf("Age = %d; want 15", ageErr.Age)
    }
}
```

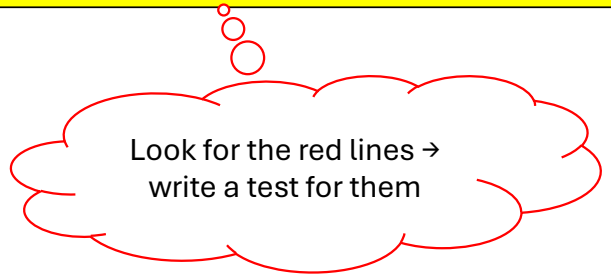
Needs import "errors"

Test coverage

- Coverage = how many % of your code is executed by your tests
- **go test -cover** → shows the percentage
- **go tool cover -html=cover.out** → opens your code in the browser
  - green = tested
  - red = never executed by a test
- Careful: 100% only means every line ran, not that every result was checked!

```
$ go test -cover
PASS
coverage: 75.0% of statements
ok      example.com/bank   0.002s
```

```
$ go test -coverprofile=cover.out
$ go tool cover -html=cover.out
```



Exercise

Take your withdraw code from Go 4 and test it!

1. Create a folder bank and run go mod init example.com/bank
2. Put WithdrawalError and withdraw(balance, amount int) error into bank.go
3. Create bank_test.go
4. Write a table-driven test TestWithdraw with these cases:
   - valid: balance 100, amount 50 → no error
   - negative: balance 100, amount -20 → error
   - insufficient: balance 100, amount 150 → error
   - limit: balance 2000, amount 1500 → error

   Tip: add a field wantErr bool to your test table
5. Bonus: use errors.As to check that Reason is the right one
6. Run go test -cover – which lines are still red and why?

Expected Output:

```
$ go test -v
=== RUN   TestWithdraw
=== RUN   TestWithdraw/valid
=== RUN   TestWithdraw/negative
=== RUN   TestWithdraw/insufficient
=== RUN   TestWithdraw/limit
--- PASS: TestWithdraw (0.00s)
    --- PASS: TestWithdraw/valid
    --- PASS: TestWithdraw/negative
    --- PASS: TestWithdraw/insufficient
    --- PASS: TestWithdraw/limit
PASS
ok      example.com/bank
```